# Class diagram - Design Decisions Document

## 1. Two-Tiered Mutual Exclusivity (Double dc Constraints)

The diagram uses a strict, deep inheritance structure with two consecutive **Disjoint/Complete (dc)** constraints. This enforces a very specific business rule:

- A Goal must be exactly either a Subgoal or a MainGoal, it cannot be a generic goal, and it cannot be both.
- If it is a MainGoal, the system forces another strict branch: it must be explicitly categorized as either a ChallengeGoal (tied to community events) or a PersonalGoal (private to the user).

## 2. Multiplicity-Constrained Self-Aggregation (0..3)

Instead of an open-ended tree structure where goals can have infinite subtasks, the design choice explicitly limits a MainGoal to a maximum of 3 subgoals (0..3 on the <is part of relationship). This represents a deliberate behavioral design decision to enforce productivity best practices (preventing users from overwhelming themselves with massive task lists).

## 3. Date-Range Composite Primary Keys for Timeblocking

Instead of using a standard auto-incrementing integer ID for scheduling, the Timeslot entity uses a **composite primary key** made of start-date and end-date.

- **Design Implication:** This prevents duplicate identical time slots from existing in the system and treats a specific window of time as a unique, reusable entity that Activity anchors to via composition.

## 4. Dual-Pointer Friendship Association

To handle friendships, the diagram uses a dedicated Friendship association table mapping a User to another User via friend1 and friend2 attributes. By shifting the relationship status (accepted: boolean) to this intersection table, the design avoids modifying the core User entity when social states change and allows for directional or bi-directional relationship tracking.

## 5. Architectural Separation of User State via Heavy Composition

The design aggressively uses strong composition (solid black diamonds) to break the user profile into specialized subsystems (GameProperties, UserPreferences, IdealDay).

- **Design Implication:** Because these lifecycle-dependent entities share the exact same primary key (suser-id / user-id acting as both PK and FK), the system ensures that deleting a user instantly purges their settings, daily routines, and gamification stats across the entire database without needing complex cascading logic or manual

cleanups.

## 6. Explicit "Creator" Separation in Shared Activities

In the `Activity` class, there is a `creator: User <<FK>>` attribute alongside the standard relationships to the `User`.

- **Design Implication:** This indicates a design choice to support shared or collaborative activities. It allows the system to distinguish between the user who is currently performing/viewing the activity and the user who originally authored it (e.g., a friend sharing a workout routine or a community creator pushing a task).